

1. What are the advantages of Polymorphism?

- Khả năng tái sử dụng mã: Có thể viết các đoạn mã chung xử lý các đối tượng khác nhau miễn là chúng có chung một lớp cha hoặc giao diện.
- Linh hoạt: Cho phép phương thức nhận tham số là lớp cha nhưng có thể hoạt động chính xác với đối tượng lớp con bất kỳ được truyền vào.
- Dễ bảo trì và mở rộng: Khi cần thêm một loại đối tượng mới vào hệ thống, chỉ cần tạo lớp con mới kế thừa từ cha và ghi đè phương thức.

2. How is Inheritance useful to achieve Polymorphism in Java?

Tính kế thừa cung cấp nền tảng cấu trúc (mối quan hệ "IS-A") cần thiết để hình có thể hoạt động:

- Cho phép tham chiếu đa hình: Nhờ kế thừa, một biến tham chiếu có kiểu là lớp cha có thể trỏ tới các đối tượng của lớp con.
- Ghi đè phương thức: Kế thừa cho phép các lớp con định nghĩa lại các hành vi của lớp cha. Nhờ đó, khi một phương thức được gọi thông qua biến tham chiếu của lớp cha, máy ảo Java sẽ tự động tìm và chạy đúng phương thức đã được ghi đè của đối tượng thực tế ở lớp con tại thời điểm chạy.

3. What are the differences between Polymorphism and Inheritance in Java?

- Bản chất: Kế thừa là khái niệm về mặt cấu trúc (tạo ra một lớp mới dựa trên một lớp đã có). Đa hình là khái niệm về mặt hành vi (một tên gọi/hàm nhưng có nhiều cách thực thi khác nhau).
- Mục đích chính: Kế thừa nhằm mục đích tái sử dụng code và xây dựng cây phân cấp đối tượng. Đa hình nhằm mục đích tạo ra giao diện lập trình thống nhất và linh hoạt, giúp tách biệt "những gì cần làm" với "cách thực hiện cụ thể".
- Mối quan hệ: Kế thừa tạo ra mối quan hệ cha - con. Đa hình là ứng dụng/kết quả của mối quan hệ cha - con đó (thường kết hợp với ghi đè phương thức) để thể hiện các hành vi đa dạng ở thời gian chạy.